PROJECT DOCUMENTATION REPORT

CloudClean

Intelligent Cloud Storage Optimization & Lifecycle Management Platform

Course: Cloud Computing | Department: CSE-AIML

PREPARED BY

Hanil Shah | Harsh Patel | Tanaya Hartalekar | Shravani Dixit

GUIDED BY

Prof. Sneha Patil

DOCUMENT DATE

September 10, 2026

Status: Implemented & tested end-to-end · Runs free & open-source, locally or via Docker

1. Project Overview

CloudClean is an intelligent Google Drive storage optimization and lifecycle-management platform. It connects to a user's Google Drive, analyzes file metadata, and produces explainable, confidence-scored recommendations for recovering storage - duplicate removal, compression, and archiving - while never taking a destructive action without explicit user confirmation. This document describes what was proposed, what has actually been built and verified, and the current status of the project.

1.1 Problem Statement

- Rapid growth of cloud data leads to storage exhaustion.
- Duplicate, outdated, and large files occupy unnecessary space.
- Users manually search and organize files, which is time-consuming.
- Existing cloud platforms provide storage statistics but very limited optimization assistance.
- Many users purchase additional storage despite having recoverable space already on their Drive.

1.2 Objectives

- Analyze Google Drive metadata efficiently (paginated, metadata-first, no unnecessary downloads).
- Detect duplicate, large, and potentially inactive files using explainable rules.
- Provide real, on-demand compression and safe archiving - not a simulated or promised result.
- Never delete data automatically; every destructive action requires explicit, informed confirmation.
- Ship entirely on free and open-source technology, runnable at zero cost beyond the user's own Google account.

2. System Architecture

CloudClean follows a metadata-first pipeline: Google Drive → Metadata Collector → File Classification → Duplicate Detection → Large/Inactive File Analysis → Compression-Potential Estimation → Recommendation Engine → SQLite cache → Dashboard. Nothing on the user's Drive changes until an action is confirmed.

Layer	Implementation	Responsibility
Client / UI	React 18 + Vite + Tailwind CSS + Recharts	Dashboard, file explorer, duplicate review, compression review, analytics, history, settings
Auth & API	Google OAuth 2.0 + FastAPI + Starlette sessions	Secure sign-in, signed-cookie sessions, REST API surface
Processing Layer	Python analyzers + RecommendationEngine	Duplicate/large/inactive detection, compression estimation, explainable scoring
Data Layer	SQLite + SQLAlchemy ORM	Caches metadata, recommendations, action history; no file content ever stored
Analytics & Dashboard	Recharts + FastAPI analytics endpoints	Storage breakdown, optimization score, real historical snapshots

2.1 Architecture Workflow

- **Authentication** - user signs in with Google OAuth, granting the minimum Drive access CloudClean needs.

- **Metadata Collection** - Drive API v3 is paged for file metadata only; no file content is downloaded at this stage.
- **Storage Analysis** - analyzers flag duplicate, large, and potentially inactive files, and estimate compression potential.
- **Recommendation Engine** - rule-based logic scores and explains each suggestion, with a stated confidence and risk level.
- **Optimize & Refresh** - on confirmation, CloudClean acts safely and then re-queries Drive so the dashboard reflects the real, current state.

3. Technology Stack

Category	Technologies Used
Frontend	React 18, Vite, Tailwind CSS, Recharts, React Router
Backend	Python 3, FastAPI, Uvicorn, Pydantic
Google Integration	Google OAuth 2.0, Google Drive API v3 (google-api-python-client)
Data	SQLite, SQLAlchemy ORM
File Processing	Pillow (images), pikepdf / qpdf (PDF), zipfile (archives, stdlib)
Testing	Pytest, FastAPI TestClient
DevOps	Docker, Docker Compose

No paid dependency is used anywhere in the stack - no paid AI/LLM API, no paid database, no paid authentication platform, no paid compression service. The application is designed to run at zero software cost.

4. Database Design

SQLAlchemy models, all indexed on the lookup fields they are queried by:

Model	Purpose
User	Account identity, OAuth tokens, Demo Mode flag
UserSettings	Per-user thresholds: large/inactive file limits, JPEG quality, minimum saving %
DriveFile	Normalized file metadata, classification, duplicate group, recommendation, risk
DuplicateGroup	A detected group of duplicate files and the recommended file to keep
Recommendation	One explainable suggestion: type, reason, saving, confidence, risk, status
OptimizationAction	Persisted history of every compress/archive/trash/restore action
AnalysisRun	Snapshot of one full analysis: files scanned, score, breakdown
StorageSnapshot	Point-in-time used/quota/recoverable bytes, used to chart real history over time

5. Features Implemented

Feature	Description
Duplicate Detection	Three-level matching: SHA-256 content hash, size + normalized filename, and filename alone. The engine recommends which copy to keep (most recently modified); the user can always override before anything is trashed.
Real Compression Engine	Pillow re-encodes JPEG/PNG/WebP; pikepdf performs real PDF stream compression. Every byte count shown is measured from the actual output, not estimated after the fact.
Verify-Before-Trash Workflow	The original file is downloaded, compressed in memory, the optimized copy is uploaded, the upload is verified with a follow-up API call, and only then is the original moved to Trash.
Explainable Recommendation Engine	Every recommendation states its type, a full plain-language reason, an estimated saving, a confidence score, and a risk level - never a black-box suggestion.
Transparent Optimization Score	A 0-100 score computed as 100 minus a set of capped, weighted, documented penalties (duplicate inefficiency, compression opportunity, inactive data, large-file concentration). No model inference is involved.
Storage Savings Calculator	Reads a JSON pricing configuration (editable without a code change) and reports an honestly-labelled "effective equivalent cost" per additional GB - never a fabricated per-MB billing rate.
Archive Workflow	Bundles selected files (single or multi-select) into a ZIP, uploads it, verifies the upload, and only then moves the originals to Trash.
Demo Mode	A deterministic, realistic simulated Drive - usable with zero Google account, zero setup, and zero cost, clearly labelled throughout the UI.
Action History & Restore	Every action (compressed, archived, trashed, restored) is persisted with before/after sizes and a timestamp; trashed files can be restored.
File Explorer	Search, filter (large / duplicate / compressible / inactive / archive candidate / already optimized), sort, pagination, and multi-select batch actions.
Analytics Dashboard	Storage-by-type, largest files, inactive-storage buckets, and a real usage-over-time chart - explicitly shows "not enough history yet" rather than fabricating data points.
Settings	Configurable large/very-large/huge file thresholds, inactive-file threshold (90/180/365/730 days), JPEG quality, and minimum-saving-to-recommend percentage.

6. Backend API Surface

Area	Endpoints
Auth	GET /auth/google/url, GET /auth/callback, POST /auth/demo, POST /auth/logout, GET /api/user
Storage	GET /api/storage, GET /api/savings
Analysis	POST /api/analyze, GET /api/analysis
Recommendations	GET /api/recommendations
Files	GET /api/files (search/filter/sort/paginate), GET /api/files/{id}
Duplicates	GET /api/duplicates, POST /api/duplicates/{group_id}/trash
Compression	GET /api/compression/candidates, POST /api/compression/{file_id}
Archive	POST /api/archive
History	GET /api/history, POST /api/history/{action_id}/restore
Settings	GET /api/settings, PUT /api/settings
Analytics	GET /api/analytics/breakdown, GET /api/analytics/history

All request/response bodies are typed with Pydantic schemas; routes are separated from services, repositories, models, and integrations rather than living in a single file.

7. Explainable Recommendations & Optimization Score

Example recommendation, captured verbatim from a live analysis run (not illustrative text):

```
Compression recommended because:
• File type: PDF
• Current size: 96.9 MB
• Estimated optimized size: 67.9 MB
• Estimated saving: 29.1 MB
• File has not been modified for 19 months
• Compression confidence: Medium
• Method: pdf_optimize - stream compression, duplicate-object removal, content untouched
```

The Optimization Score is **100 minus a set of capped, weighted penalties**, each a documented constant applied to that factor's share of total storage (duplicate inefficiency, compression opportunity, inactive data, large-file concentration). A real run scored **73/100** with the breakdown: duplicate inefficiency -2.3, compression opportunity -1.4, inactive data -8.2, large-file concentration -15.0 points.

8. Safety, Security & Privacy

8.1 Safety Guarantees

- Nothing is ever permanently deleted automatically - the most destructive action CloudClean takes is moving a file to Google Drive's own Trash.
- Compression: original is only trashed after the optimized copy is uploaded **and** verified to exist on Drive.
- Duplicates: the user always confirms which copy to keep before anything is trashed; the engine's pick is only a default.

- If a compression or archive operation would not save a meaningful amount, the original is left untouched.

8.2 Security

- OAuth credentials live only in environment variables - never in frontend code or version control.
- Sessions use Starlette's signed-cookie SessionMiddleware with SameSite=Lax, giving baseline CSRF protection.
- ZIP archive entry names are sanitized (path separators stripped) before being written, preventing path-traversal on extraction.
- Unhandled exceptions are logged server-side and returned to the client as a generic message - no stack traces are exposed.

8.3 Privacy

- File content is never written to disk. Downloads, compression, and ZIP creation all happen in memory and are discarded once the request completes.
- The database stores only metadata, recommendation text, hashes where needed, and action history - never file content.

9. Testing & Verification

22 automated backend tests (pytest) are passing, covering: duplicate detection at all three match levels, inactivity/size-tier rules, compression-potential estimation, the recommendation engine's explainability and score bounds, **real** compression round-trips (a JPEG is genuinely re-encoded and re-decoded, a PDF's page count survives optimization, ZIP entries are sanitized), and a full integration test that drives the actual HTTP API through Connect → Analyze → Recommend → Optimize → Refresh against Demo Mode.

The frontend was verified manually in a real browser end-to-end: Demo Mode connect, Analyze Drive, reviewing explainable recommendations, confirming a duplicate trash, running a real PDF compression (contract.pdf: 97 MB → 70 MB, saved 26 MB / 27.3%), a multi-select batch archive (2 files, 3.90 GB → saved 1.22 GB), and viewing the resulting Action History with a working Restore action.

10. Demo Mode vs. a Real Google Drive Connection

CloudClean supports two modes side by side, and they are never mixed in the same session:

	Demo Mode	Real Google Drive
Setup required	None - one click, no account	Free Google Cloud project + OAuth client (see Section 11)
Data	Deterministic, realistic simulated files	The user's actual Drive metadata, via Drive API v3
Actions	Fully simulated - safe to try everything	Real API calls: real compression, real Trash, real uploads
Cost	$0	$0 within Google's free API quota (see Section 11)

Yes - integrating with a real Google Drive is fully possible at no cost. The backend already implements the complete real OAuth 2.0 + Drive API v3 flow (token exchange, refresh, metadata paging, download, upload, trash, restore). Turning it on requires only a free Google Cloud project and a free OAuth client - no Google API

billing account is required for this usage pattern, and Drive API's free daily quota is generous enough for personal use. Section 11 gives the exact steps.

11. Connecting a Real Google Drive (Free Setup)

- Create a project at console.cloud.google.com (free).
- Enable the **Google Drive API** under APIs & Services → Library.
- Configure the OAuth consent screen (External or Internal), add the Drive/userinfo scopes, and add your own account as a test user if the app is in Testing mode.
- Create an OAuth Client ID (Web application) with redirect URI `http://localhost:8000/auth/callback`.
- Copy the Client ID and Secret into `.env` (`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`) and generate a `SECRET_KEY`.
- Restart the backend - the "Connect Google Drive" button now redirects to Google's real consent screen.

CloudClean requests the broad `drive` scope rather than `drive.file`, because it needs to read metadata and move to Trash files it did not itself create - this is disclosed to the user on the Connect page before they authorize.

12. Current Project Status

12.1 Completed

- Full-stack scaffolding: React/Vite frontend, FastAPI backend, SQLite database, Docker configuration.
- Real Google OAuth 2.0 + Drive API v3 integration (metadata paging, download, upload, trash, restore).
- Metadata-first analysis pipeline: classification, duplicate detection, large/inactive-file rules, compression estimation.
- Rule-based, fully explainable Recommendation Engine and transparent Optimization Score.
- Real compression engine (JPEG/PNG/WebP via Pillow, PDF via pikepdf) and ZIP archiving.
- Safe, verify-before-trash and confirm-before-trash workflows for compression, duplicates, and archiving.
- Dashboard, File Explorer, Duplicate Review, Compression Review, Analytics, History, and Settings pages.
- Demo Mode with a deterministic, realistic simulated Drive.
- 22 automated backend tests, and a full manual browser walkthrough of every core flow.
- Complete README with setup, Google Cloud configuration steps, security notes, and known limitations.

12.2 Not Yet Done / Deliberately Deferred

- Background job queue with live progress for very large Drives (current analysis runs synchronously, adequate for personal-Drive scale).
- Automated Playwright/browser test suite (currently: pytest + a full manual browser verification pass).
- Multi-cloud support (OneDrive, Dropbox, S3) - noted as future scope, not part of the current build.
- Production deployment (hosting, HTTPS, a non-default SECRET_KEY) - the project currently targets local/localhost use plus Docker Compose.

13. Known Limitations

- Google Drive storage is shared with Gmail and Google Photos - CloudClean only sees and affects Drive files.
- Not every file can be meaningfully compressed; already-compressed formats are explicitly flagged rather than falsely promised savings.
- Google Workspace-native files (Docs/Sheets/Slides) have no binary size and cannot be compressed like ordinary files.
- Compression estimates shown before an action can differ from the actual result - the UI always distinguishes "estimated" from "actual".
- Moving a file to Trash does not necessarily reclaim quota instantly in every account configuration.
- Google API quotas/rate limits can affect very large Drives; CloudClean pages through listings rather than fetching everything at once.
- A file being old is never treated as proof it is unnecessary - the app always uses "potentially inactive" language and offers archiving, not deletion.

14. Future Scope

- **Multi-Cloud Support** - extend analysis and actions to OneDrive, Dropbox, and AWS S3.
- **Optional Lightweight ML** - pattern-based personalization layered on top of the existing rule-based engine, without losing explainability.
- **Background Job Queue** - asynchronous processing with live progress for very large Drives.
- **Scheduled Re-Analysis** - periodic automatic analysis with a digest of new recommendations.
- **Production Deployment** - HTTPS, managed secrets, and a hosted deployment guide.

15. Conclusion

CloudClean was built and verified end-to-end, not merely designed. Every core capability described in the original proposal - duplicate detection, compression, lifecycle management, and storage insight - is implemented with real logic behind it: real OAuth, a real Drive API integration, a real compression engine, and a rule-based recommendation engine whose every output is explainable. The project also ships a zero-cost Demo Mode so the full experience can be evaluated without connecting a real account, and the entire stack runs on free, open-source technology.

16. References (Literature Survey)

Paper	Year	Key Contribution
Helping Users Automatically Find and Manage Sensitive, Expendable Files in Cloud Storage	2020	ML on metadata + content features predicts desired file management with 79% accuracy.
Reducing Latency and Storage Costs in Cloud Applications Through Advanced Data Management	2025	Automated cleanup via regex, lifecycle rules, and fuzzy-matching deduplication.
Leveraging Data Deduplication Approaches in Cloud Storage: A Succinct Review	2025	Survey of approaches for identifying and eliminating redundant cloud data.
Building a Data Management System for the Cloud: Lessons Learned and Future Directions	2025	Lessons from building Snowflake: query execution, file pruning, layout optimization at scale.